



Learning Roadmap

AI Engineer Roadmap: Backend (70%) + LLM Engineering (30%)

Duration: 1 - 1.5 years for the core Backend + LLM track. Add 2-3 months if pursuing the optional Frontend Engineering phase for full-stack breadth.

Philosophy: Every section follows **Concept** → **Pattern** → **Tool**. You should be able to explain *why* a pattern exists and what problem it solves before you touch the library that implements it. Tools change every 2 years, patterns don't.

How to use this doc

For every topic, ask three questions in order:

1. **What problem does this solve?** (the "why")
 2. **What's the general shape of the solution?** (the pattern, framework agnostic)
 3. **What's one popular tool that implements this today?** (pick one, go deep, don't tool-hop)
-

Phase 1: API & Request Lifecycle (Month 1-2)

Concepts first

- What actually happens between a client sending a request and a server responding
 - Socket accepts connection → HTTP parsing → routing → middleware chain → handler → response serialization
- Why layered architecture exists (separation of concerns, testability, swappability)

- **Controller:** maps URL/verb → function, handles request/response shape only
- **Service:** business logic, orchestrates repositories, framework-agnostic
- **Repository/ORM layer:** abstracts data access so the service doesn't care if it's Postgres or Mongo
- Middleware as a *chain of responsibility* pattern (auth, logging, rate-limit, error handling)
- API versioning strategies and *why* you need them (breaking changes without breaking existing clients)
 - URI versioning, header versioning, content negotiation: know the tradeoffs of each
- Caching as a pattern, not a Redis feature
 - Cache-aside, write-through, write-behind, TTL vs eviction (LRU/LFU)
 - Where caching lives: client, CDN, API gateway, application layer, DB query cache
- REST vs GraphQL: not "which is better" but *what problem each optimizes for* (over/under-fetching, contract rigidity, caching implications)
- **Client-server communication patterns beyond simple request/response:** this is where most junior engineers only know HTTP request/response and stop, but real systems need different shapes depending on who needs to know what, and how fast.
 - **Polling:** client repeatedly asks "anything new?", simplest pattern, wasteful at scale
 - **Long polling:** server holds the request open until it has data, reduces wasted round-trips, still one-shot per update
 - **SSE (Server-Sent Events):** one-way, server pushes a stream of events to the client over a single long-lived HTTP connection. Good fit for notifications, live feeds, and LLM token streaming, anywhere the server needs to keep talking but the client doesn't need to talk back

- **WebSockets:** full-duplex, persistent connection, both sides can push at any time. Fit for chat apps, collaborative editing, multiplayer, anywhere low-latency two-way communication matters
- **gRPC / HTTP streaming:** worth knowing conceptually as the "typed, binary, streaming" alternative to REST, common in service-to-service communication
- The pattern to internalize: match the communication shape (request/response, one-way stream, or bidirectional stream) to what the interaction actually needs, don't default to WebSockets for everything just because it's flexible
- Testing pyramid: unit vs integration vs e2e, and why the ratio matters (fast feedback vs confidence)

Then pick tools

- One framework end-to-end (e.g., FastAPI/Express/Spring): build a CRUD API with proper layering
- One testing framework (pytest/Jest/JUnit)
- Postman/Insomnia/Bruno for manual testing, then move to automated
- Native `EventSource` (SSE) and the `ws` library or framework-native WebSocket support, to see both real-time patterns hands-on

Checkpoint project

Build a small API (e.g., a "todo" or "bookmarks" service) with controller/service/repository separation, versioned endpoints, one caching layer, and a test suite. Deliberately swap the DB or ORM afterward to prove the layering worked. Add one SSE endpoint (e.g., streaming live updates) and one WebSocket endpoint (e.g., a live counter or chat) so you've felt the difference between one-way and two-way real-time communication firsthand.

Phase 2: Databases (Month 2-4)

Concepts first

- SQL vs NoSQL is a decision tree, not a preference; understand what each family trades off
 - **Relational**: ACID, joins, strong consistency
 - **KV (Redis)**: speed, simple access patterns, ephemeral/session data
 - **Document (MongoDB, DynamoDB)**: flexible schema, denormalized reads
 - **Columnar (Cassandra)**: write-heavy, time-series, wide-row analytics
- Schema planning: normalization vs denormalization, and *when* to break normal form on purpose
- Indexing: B-tree vs hash indexes, composite indexes, why an index speeds reads but costs writes
- Triggers: DB-level reactive logic, and why you'd choose this over application-level logic (and why you often shouldn't)
- RLS (Row-Level Security): enforcing access control at the data layer instead of trusting every service to get it right
- JSONB: the "flexible schema inside a rigid schema" pattern; when semi-structured data belongs in a relational DB vs a document store

Then pick tools

- Postgres (covers relational + JSONB + RLS in one tool, efficient choice)
- Redis (KV)
- One of Mongo/DynamoDB (document)
- Optional: Cassandra if you want columnar exposure, otherwise just understand it conceptually

Checkpoint project

Take the API from Phase 1, add proper indexing, write a migration, add an RLS policy, and add a Redis cache layer in front of a hot read path. Measure the difference with/without index and with/without cache.

Phase 3: Observability (Month 4-5)

Concepts first

- The three pillars: **Logs** (discrete events), **Metrics** (aggregated numbers over time), **Traces** (a request's journey across services)
- Why OTEL (OpenTelemetry) exists as a *vendor-neutral standard*: instrument once, send anywhere
- Cardinality, sampling, and why "log everything" doesn't scale
- The difference between monitoring (known-unknowns, dashboards/alerts) and observability (unknown-unknowns, ability to ask new questions of existing data)

Then pick tools

- Instrument with OTEL SDK
- Pick one backend to visualize: Signoz (easiest all-in-one) or Grafana + Prometheus (metrics) as a separate combo
- Prometheus for metrics scraping, Grafana for dashboards, Signoz/Jaeger for traces: understand these are interchangeable backends for the same OTEL data

Checkpoint project

Instrument the API from Phase 1-2 with traces + metrics + logs, and build one dashboard that shows request latency, error rate, and DB query time.

Phase 4: Async Communication Patterns (Month 5-7)

(This phase covers service-to-service and event-driven communication. Phase 1 already covered client-server real-time patterns like SSE and WebSockets, this phase is about how backend systems talk to each other.)

Concepts first

- **Sync vs async**: when a caller needs an immediate answer vs when "fire and forget" is acceptable, and the failure modes each introduces
- **Event-driven design**

- Instead of Create/Update/Delete, model everything as an immutable **append-only log of events** (Create-only from the DB's perspective; updates/deletes are just new events)
- Example: `user 101 created todo 202` → `user 101 updated todo 202` → final state is derived by **replaying the log**
- Why this matters: auditability, replayability, temporal debugging ("what did the state look like at time T")
- This is the conceptual root of **event sourcing**, worth explicitly naming and reading about once the pattern clicks
- **Pub-Sub**: decoupling producers from consumers, one event, many independent subscribers
- **Event bus**: the architectural component that routes events between producers/consumers, can be as simple as an in-memory bus or as heavy as Kafka
- Delivery guarantees: at-most-once vs at-least-once vs exactly-once, and why idempotency matters once you accept "exactly-once" is often a myth
- Dead-letter queues: what happens when a consumer keeps failing on a message

Then pick tools

- Kafka (covers event-driven, pub-sub, and event bus in one tool)
- Optional: RabbitMQ for pure queue/pub-sub comparison, to see the same patterns implemented differently

Checkpoint project

Take an action from your API (e.g., "todo created") and emit it as an event to Kafka. Build a consumer that replays events to reconstruct current state, independent of the source DB.

Phase 5: Cloud Engineering (Month 7-9)

Every backend engineer needs to know where their code actually runs and what it costs to run it there. Same rule: understand the abstraction ladder before picking a provider.

Concepts first

- **The compute abstraction ladder:** bare metal → virtual machines → containers → serverless functions. Each step trades control for convenience, know what you give up and gain at each step
 - **VMs:** full OS control, you manage everything above the hypervisor, good for long-running, stateful, or unusual workloads
 - **Containers:** package the app + its dependencies together (Docker as the packaging pattern), portable across environments, still need something to run and schedule them
 - **Serverless / FaaS:** you provide a function, the platform handles provisioning, scaling to zero, and billing per invocation. Concept to internalize: statelessness is a requirement, not a suggestion, and cold starts are the tradeoff for not paying when idle
- **Storage types and when backend systems need each**
 - **Object storage** (e.g., S3): unstructured blobs, images, backups, cheap and durable, no filesystem semantics
 - **Block storage** (e.g., EBS): attached disks for VMs/databases, needed when you need filesystem-level access
 - **File storage** (e.g., EFS): shared filesystem across multiple compute instances
- **Networking fundamentals:** VPCs and subnets (isolating your infrastructure), security groups/firewalls (who can talk to whom), load balancers (tie this back to Phase 5's system design content), DNS
- **Container orchestration as a pattern:** Kubernetes's job is scheduling, self-healing, and scaling containers, understand the concept (declarative desired state, a control loop that reconciles reality to match it) even if you start with a simpler managed container service

- **Infrastructure as Code (IaC):** infrastructure defined declaratively in version-controlled files instead of clicked together manually, why this matters for reproducibility and rollback
- **CI/CD as a pattern:** build → test → deploy pipeline, and deployment strategies (blue-green, canary, rolling) as ways to reduce the blast radius of a bad release
- **Secrets and config management:** why credentials never belong in code or images, environment-based configuration
- **Cost and scaling tradeoffs:** reserved vs on-demand pricing, autoscaling policies, and the operational cost of "just use serverless for everything" vs "just use one big VM"

Then pick tools

- One cloud provider to go deep on (AWS is the most common default, GCP/Azure are fine alternatives, don't split time across all three early on)
- Docker for containers
- One managed container/serverless platform (e.g., AWS Lambda or ECS/Fargate, or Cloud Run on GCP) rather than self-managing Kubernetes on day one
- Terraform for IaC (cloud-agnostic, transferable skill)
- GitHub Actions (or your provider's native CI/CD) for the pipeline

Checkpoint project

Take the API from Phase 1-4 and deploy it two ways: once as a container on a managed container service, and once as a serverless function. Put a load balancer in front, write the infrastructure in Terraform, and wire up a CI/CD pipeline that runs your Phase 1 test suite before every deploy. Compare cold-start latency and cost behavior between the two approaches.

Phase 6: System Design Synthesis (Month 9-10)

(This is where the patterns from every prior phase get stress-tested together.)

- Horizontal vs vertical scaling
- Load balancing strategies (round robin, least connections, consistent hashing)
- Rate limiting patterns (token bucket, leaky bucket, sliding window)
- Circuit breakers, retries with backoff, bulkheads: failure isolation patterns
- CAP theorem: in plain terms, what you actually give up under a network partition
- Idempotency keys in API design (ties back to Phase 4's delivery guarantees)

Checkpoint project

Design (on paper/whiteboard, doesn't need to be built) a system combining everything so far (e.g., a URL shortener or notification service) and defend your choices out loud. This is the interview-readiness checkpoint.

Phase 7: LLM Engineering (Month 10-14, fully hands-on)

No model training, no PyTorch/TF internals. This is about **building systems on top of LLMs**, same pattern-first philosophy applies.

Concepts first, tools second: same discipline as backend

1. Prompting as an interface, not a trick

- Concept: the prompt is the API contract with a probabilistic system
- Zero-shot vs few-shot, system vs user role separation, structured output (JSON mode/schemas)
- Tool: whichever model API you're already using (Claude/OpenAI) via their SDK

2. RAG (Retrieval-Augmented Generation)

- Concept: LLMs have a context window, not a database; RAG is the pattern for injecting relevant knowledge at request time instead of retraining
- Sub-patterns: chunking strategy, embeddings, similarity search, re-ranking, chunk-size/overlap tradeoffs

- Tools: an embedding model + a vector store (pgvector if you want to reuse Postgres skills, or a dedicated one like Chroma/Pinecone)

3. Agents & tool use

- Concept: an LLM that can decide *when* to call external functions/tools instead of just generating text: the ReAct pattern (reason → act → observe → repeat)
- Function calling / tool use as the mechanism, planning/orchestration as the pattern above it
- Multi-agent patterns (orchestrator-worker, hand-offs) once single-agent is solid
- Tools: native function calling in Claude/OpenAI APIs, then optionally a framework (LangChain/LlamaIndex), **after** you've built one agent loop by hand so the framework's abstractions aren't magic

4. Context & memory management

- Concept: context window is a finite, expensive resource, same mental model as caching from Phase 1
- Short-term (conversation window) vs long-term (retrieved memory): this is RAG applied to "memory" instead of "documents"

5. Evaluation & guardrails

- Concept: how do you know your LLM system is actually working, this is the "testing" phase for LLM apps
- Building eval sets, LLM-as-judge, regression testing prompts, hallucination detection
- Guardrails: input validation, output validation, PII filtering

6. LLM-Ops (the "backend" of LLM engineering, this is where your 70% pays off)

- Latency/cost tradeoffs (model choice, prompt caching, streaming)
- Semantic caching (same cache-aside pattern from Phase 1, applied to LLM responses)

- Observability for LLM calls (same OTEL mindset from Phase 3: trace token usage, latency, cost per request)
- Rate limiting and retries against LLM provider APIs (same patterns from Phase 6)
- Streaming responses to the client over SSE, the LLM use case that makes Phase 1's SSE section click into place

Checkpoint projects (in order)

1. A RAG-based Q&A system over a small document set: build chunking/embedding/retrieval by hand once before using a framework
2. A single tool-using agent (e.g., an agent that can query your Phase 1-4 API): hand-rolled function-calling loop first
3. A small eval harness that scores your agent's outputs against a test set
4. Wire observability (Phase 3 skills) into the LLM app: trace latency, token cost, and cache hit rate per request. Stream the agent's responses to a client over SSE instead of waiting for the full response.

Phase 8 (Optional): Frontend Engineering (add 2-3 months)

For engineers who want full-stack breadth on top of the core track. Same rule applies: understand the browser and the rendering pattern before picking a framework.

Concepts first

- **Browser rendering pipeline:** HTML parsed into the DOM, CSS parsed into the CSSOM, combined into a render tree, then layout and paint. Understanding this explains why certain CSS/JS changes are cheap and others trigger expensive reflows.
- **Component-based UI:** breaking a UI into small, reusable, self-contained pieces, the frontend equivalent of the controller/service/repository separation from Phase 1.

- **State management patterns:** local component state vs global/shared state, unidirectional data flow (state flows down, events flow up), and why this makes UIs predictable to debug.
- **Client-server data patterns:** client-side rendering (CSR) vs server-side rendering (SSR) vs static site generation (SSG), and the tradeoff each makes between time-to-first-byte, interactivity, and SEO.
- **Data fetching & caching on the client:** request deduplication, stale-while-revalidate, optimistic updates, this is the same caching mindset from Phase 1, applied client-side.
- **Consuming real-time backends:** how a frontend actually consumes the SSE and WebSocket endpoints from Phase 1, `EventSource` for SSE, the native `WebSocket` API (or a wrapper like `socket.io-client`) for two-way, and how to handle reconnect/backoff on the client side.
- **Routing:** client-side routing as a pattern for single-page apps vs traditional server-driven navigation.
- **Forms & validation:** controlled vs uncontrolled inputs, client-side validation as a UX layer that never replaces server-side validation.
- **Styling architecture:** utility-first CSS, component-scoped CSS, CSS-in-JS, know the tradeoffs (bundle size, maintainability, reusability) rather than defaulting to whatever's trendy.
- **Build tooling concepts:** bundling, transpilation, tree-shaking, code-splitting, why a build step exists at all between source code and what ships to the browser.
- **Accessibility (a11y):** semantic HTML, ARIA roles, keyboard navigation, this is a correctness concern, not a nice-to-have.
- **Frontend testing:** same pyramid as Phase 1, unit tests for logic, component tests for rendering behavior, e2e tests for full user flows.

Then pick tools

- One framework end-to-end (React is the most common default, Vue is a fine alternative)

- One build tool (Vite is the current standard)
- One styling approach (Tailwind CSS is a practical default to start with)
- One state management tool, but only after feeling the pain of prop-drilling by hand first (Context API or a lightweight store like Zustand/Redux)
- Testing tools: React Testing Library for components, Playwright or Cypress for e2e

Checkpoint project

Build a frontend for the API from Phase 1-4: list/detail views, a form that creates/updates data, client-side routing between views, and one place where you deliberately handle loading/error/optimistic-update states. Then connect it to the LLM app from Phase 7 (e.g., a chat UI that streams responses over SSE) so the full stack, backend, cloud, LLM, and frontend, is exercised end to end in one project.

Suggested Pacing Summary

Phase	Topic	Duration
1	APIs & request lifecycle (incl. SSE/WebSockets)	Month 1-2
2	Databases	Month 2-4
3	Observability	Month 4-5
4	Async communication patterns	Month 5-7
5	Cloud engineering	Month 7-9
6	System design synthesis	Month 9-10
7	LLM engineering (hands-on)	Month 10-14
8	Frontend engineering (optional)	Month 14-16/17

Remaining runway (Month 14-18, or Month 17-18 if Phase 8 is included): deepen whichever half is weaker, contribute to a real production system, and revisit Phase 6 system design with the cloud and LLM patterns folded in (e.g., designing a RAG system for scale and cost, not just correctness).